

## 1. Introducción

En esta cuarta parte del obligatorio, se implementó la infraestructura necesaria para ejecutar una aplicación web de gestión de tareas usando Docker. El objetivo fue construir un entorno multi-servicio, aplicando conceptos de virtualización, redes internas, persistencia y balanceo de carga.

La aplicación, desarrollada en Node.js con Express, utiliza Redis como base de datos y Nginx como balanceador de carga. Mediante Docker Compose, se definió un entorno reproducible que permite iniciar todos los servicios con un solo comando, simulando un despliegue real y aplicando prácticas de infraestructura moderna basada en contenedores.

## 2. Arquitectura Implementada

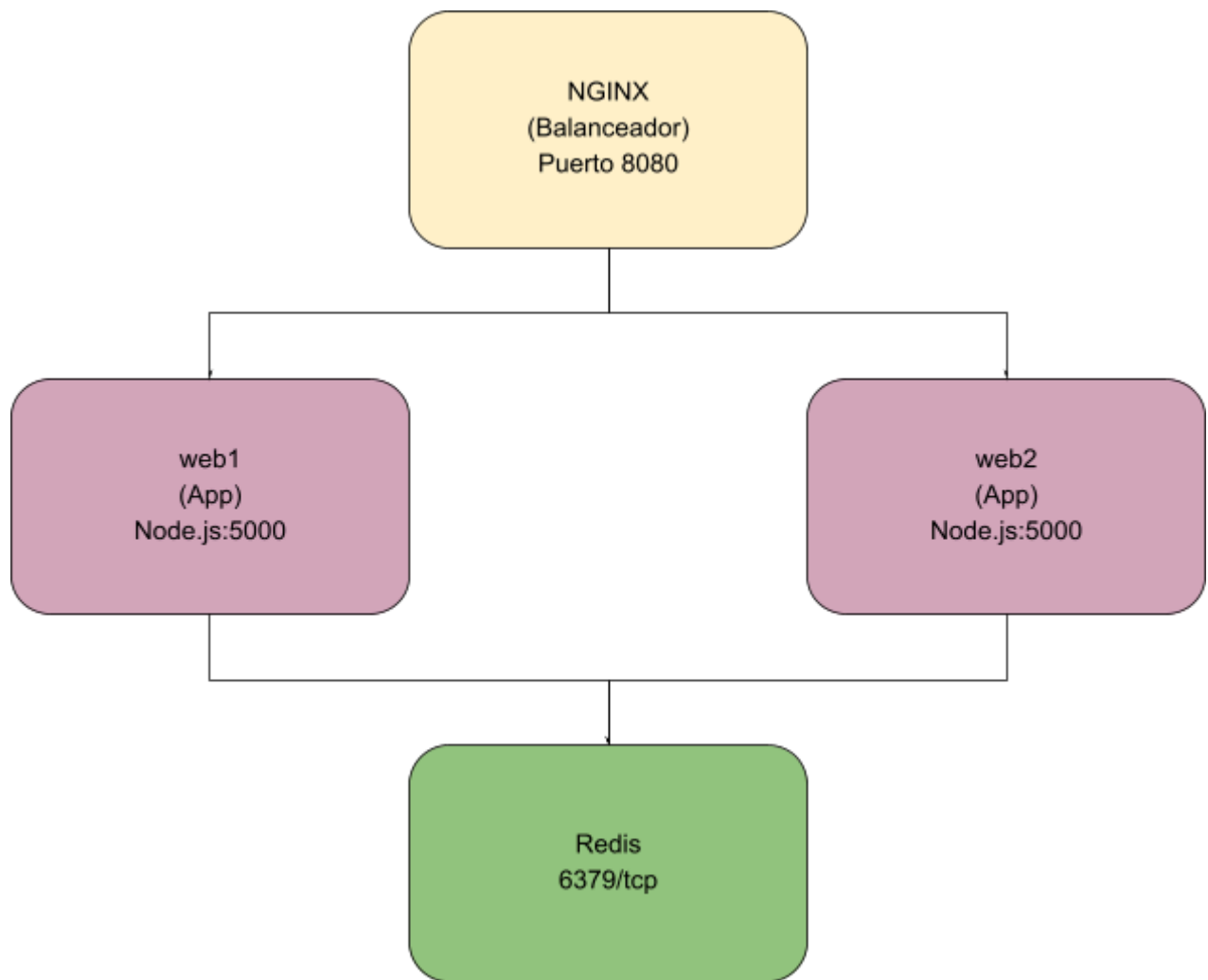
La arquitectura desarrollada está compuesta por cuatro contenedores Docker que conforman un entorno distribuido interconectado mediante una red virtual interna creada por Docker Compose. Cada servicio cumple un rol específico dentro del sistema, garantizando la separación de responsabilidades y una comunicación eficiente entre componentes.

- **web1 y web2:** dos instancias independientes de la aplicación Node.js, que ejecutan el mismo código y escuchan en el puerto interno 5000. Se encargan de procesar las peticiones HTTP, manejar la lógica de negocio y conectarse con Redis. La duplicación de instancias permite aplicar balanceo de carga y asegurar la disponibilidad del sistema.
- **nginx:** actúa como balanceador de carga y proxy inverso, recibiendo las solicitudes externas en el puerto 8080 y distribuyéndolas equitativamente entre web1 y web2 mediante la estrategia *round robin*. Además, funciona como punto de entrada único al sistema, ocultando la red interna y mejorando la seguridad al exponer solo un puerto público.
- **redis:** es la base de datos en memoria utilizada para almacenar las tareas. Se ejecuta sobre la imagen `redis:alpine` y cuenta con un volumen persistente (`redis_data`) que conserva los datos aunque el contenedor se reinicie. Este servicio no se expone al exterior, sino que es accesible únicamente por los contenedores de la aplicación a través de la red interna.

Todos los servicios se comunican dentro de la red virtual bridge `app_net`, que aísla la arquitectura del sistema operativo anfitrión.

El único puerto visible externamente es el 8080, correspondiente a Nginx, a través del cual se canalizan todas las interacciones del usuario con la aplicación.

El siguiente esquema representa la arquitectura implementada:



### 3. Decisiones Técnicas

Durante la implementación se tomaron decisiones orientadas a lograr un entorno eficiente, modular y fácil de desplegar. Cada servicio fue configurado según su función dentro de la arquitectura.

#### 3.1. Elección de Imágenes y Entorno

- **Node.js (web1 y web2):** se utilizó la imagen oficial `node:18-alpine` por su ligereza y estabilidad. La versión *alpine* reduce el tamaño de las imágenes y acelera la construcción de contenedores. Cada instancia expone el puerto 5000 y ejecuta el comando `npm start`.
- **Redis:** se empleó la imagen `redis:alpine`, con un volumen persistente (`redis_data`) para conservar los datos entre ejecuciones.
- **Nginx:** se usó la imagen `nginx:latest`, configurada mediante el archivo `nginx.conf`, con un bloque `upstream` que incluye a `web1` y `web2`, funcionando como balanceador de carga HTTP.

#### 3.2. Orquestación de Servicios

La coordinación del sistema se realizó con Docker Compose, que define los servicios en el archivo *docker-compose.yml*. Compose crea automáticamente la red interna y los volúmenes, asegurando el orden correcto de inicio mediante *depends\_on*. Solo se expone el puerto 8080 de Nginx; los demás permanecen internos dentro de la red *app\_net*.

### 3.3. Balanceo y Escalabilidad

El balanceo se configuró en Nginx con:

```
upstream loadbalancer {  
    server web1:5000;  
    server web2:5000;  
}
```

La estrategia round robin reparte las solicitudes entre las instancias y permite escalar fácilmente con: *docker compose up --scale web=3*. Esto mejora el rendimiento y la disponibilidad ante mayor carga o fallas.

### 3.4. Persistencia y Mantenimiento

Redis utiliza un volumen persistente para conservar la información aún si el contenedor se reinicia. Las imágenes se construyen de forma reproducible, garantizando que el entorno sea consistente en cualquier equipo con Docker.

## 4. Instrucciones de Ejecución y Pruebas

A continuación se describen los pasos para ejecutar y probar el sistema. Las instrucciones se basan en el uso de Docker y Docker Compose, que permiten automatizar la creación y despliegue de los servicios.

### 4.1. Requisitos previos

Antes de iniciar la ejecución se debe contar con:

- Docker Desktop o Docker Engine instalado y funcionando.
- Docker Compose versión 2.0 o superior.
- Los archivos del proyecto en la estructura:

```
obligatorio_so_parte4/  
├── app/  
│   ├── Dockerfile  
│   ├── package.json  
│   ├── server.js  
│   └── (otros archivos)  
├── nginx/  
│   ├── Dockerfile  
│   └── nginx.conf  
└── docker-compose.yml
```

### 4.2. Construcción y ejecución del sistema

Para construir las imágenes y levantar los contenedores, ejecutar en la carpeta raíz del proyecto: *docker compose up --build*

Este comando descarga las imágenes base, crea la red interna *app\_net*, el volumen *redis\_data* y levanta los servicios *redis*, *web1*, *web2* y *nginx* en el orden correspondiente. Durante la ejecución se mostrarán mensajes en consola indicando el inicio de cada servicio, por ejemplo:

```
web1 | Sistema de Gestión de Tareas escuchando en el puerto 5000
web2 | Sistema de Gestión de Tareas escuchando en el puerto 5000
nginx | Configuration complete; ready for start up
```

#### 4.3. Acceso a la aplicación

Una vez iniciado el entorno, la aplicación se encuentra disponible en el navegador en: *http://localhost:8080*. Nginx recibe las solicitudes en el puerto externo 8080 y las distribuye entre las instancias *web1* y *web2*.

#### 4.4. Verificación del funcionamiento

Para comprobar el correcto funcionamiento del sistema se puede:

- Crear o eliminar tareas desde la interfaz.
- Observar en los logs de Nginx cómo se reparten las solicitudes.
- Confirmar que Redis conserva los datos luego de reiniciar los contenedores.

Comandos útiles para verificar el estado:

*docker ps* – muestra los contenedores activos

*docker logs nginx* – consulta los registros de Nginx

*docker logs web1* – muestra la actividad de la aplicación

*docker logs redis* – confirma la conexión con la base de datos

#### 4.5. Detención del sistema

Para detener los contenedores:

- Si se ejecutó en primer plano: presionar *Ctrl + C*.
- En cualquier momento: *docker compose down*

Si se desea eliminar también los volúmenes (borrar los datos de Redis):

*docker compose down -v*

#### 4.6. Escalado y pruebas de carga

Para aumentar la cantidad de instancias y probar la escalabilidad horizontal:

*docker compose up --scale web=3*

Este comando crea una nueva instancia (*web3*) que se integra automáticamente en el balanceo de carga de Nginx, demostrando la capacidad del sistema para adaptarse a una mayor demanda sin modificar la configuración original.

## 5. Análisis de Ventajas y Desventajas

La arquitectura implementada con Docker ofrece un entorno portable, eficiente y fácil de mantener. Sin embargo, también presenta algunas limitaciones que deben considerarse al migrar a un entorno de producción.

### 5.1. Ventajas

Cada servicio se ejecuta en su propio contenedor con dependencias aisladas, lo que asegura la reproducibilidad del entorno en cualquier equipo con Docker.

El uso de Docker Compose simplifica el despliegue al permitir iniciar todos los servicios con un solo comando, automatizando la creación de redes y volúmenes.

La configuración de Nginx como balanceador posibilita la escalabilidad horizontal mediante la opción `--scale`, distribuyendo las peticiones entre las instancias disponibles y mejorando la disponibilidad del sistema.

La arquitectura mantiene una buena eficiencia gracias al balanceo de carga *round robin* y al uso de imágenes *alpine*, que reducen el tamaño de los contenedores y aceleran la construcción del entorno.

Además, la persistencia de datos se logra mediante el volumen *redis\_data*, que conserva la información aunque los contenedores se detengan o reinicien.

### 5.2. Desventajas

Redis no cuenta con autenticación y Nginx opera solo bajo HTTP, lo que limita la seguridad del sistema.

El entorno carece de monitoreo o registro centralizado, lo que complica la detección de errores o el análisis del rendimiento.

La persistencia depende únicamente del volumen local de Redis, sin mecanismos de respaldo ni replicación.

Finalmente, Docker Compose no ofrece alta disponibilidad ni escalado dinámico, por lo que resulta más adecuado para entornos de desarrollo que para producción.